

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example. Answer: Testing as an Engineering Activity Testing should be treated as a systematic engineering activity instead of doing random or ad-hoc testing. For a banking application, the testing process should begin with understanding the requirements and preparing a proper test plan. Test cases should be designed using techniques such as equivalence partitioning, boundary value analysis, decision table testing and state transition testing. Testing tools can be used for test management, automation and defect tracking, while standard procedures should be followed by all testers. For example, in a money transfer function, test cases should check valid accounts, insufficient balance, invalid account numbers and transaction limits. Regression testing should also be performed whenever changes are made. This systematic approach improves defect detection, reduces missed defects and increases the overall quality and reliability of the banking application.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes. Answer: Structured Testing Process for ATM Withdrawal	3

	A structured testing process should be followed throughout the SDLC to improve the quality of an ATM system. During requirement analysis, the tester should identify requirements such as card validation, PIN verification, balance checking and cash withdrawal. During test planning and test design, suitable test cases should be prepared for valid PINs, invalid PINs, insufficient balance, invalid withdrawal amounts and daily limits. During execution, actual results are compared with expected results and defects are recorded. After fixing the defects, retesting and regression testing should be performed. For example, if a customer has ₹5,000 and withdraws ₹2,000, the ATM should dispense ₹2,000 and update the balance to ₹3,000. Following the same process for every release reduces defects, improves consistency and makes the ATM withdrawal function more reliable.	
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples. Answer: Error, Defect and Failure Error, defect and failure are related but different concepts in software testing. An error is a human mistake made during requirement analysis, design or coding, while a defect is the problem introduced into the software because of that mistake. A failure occurs when the software executes the defective code and produces an incorrect result. For example, if a developer writes an incorrect condition for password validation, it is an error; if the login system accepts an incorrect password because of that code, it is a defect; and when a user actually enters the wrong password and is allowed to log in, it becomes a failure. Therefore, the relationship can be understood as Error → Defect → Failure. Properly understanding these terms helps testers prepare clear defect reports containing steps to reproduce, expected result, actual result and other relevant details.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example. Answer: Defect Clustering and Test Prioritization	3

	Defect clustering means that a small number of software modules may contain a large number of defects. Therefore, test planning should give more attention to modules where defects are frequently found. Previous defect data can be analyzed to identify high-risk areas and test cases for these modules can be given higher priority. Critical business functions should also be tested before less important functions. For example, in an online banking application, the payment and transaction modules may contain more defects than the profile module, so transaction-related test cases should be executed first. Other testing principles such as early testing, regression testing and the fact that exhaustive testing is not possible should also be considered. This approach makes testing more efficient because available time and resources are focused on high-risk areas, helping to detect important defects earlier.	
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements. Answer: Defect Classification and Repository A defect repository is a central place where software defects are recorded, classified and maintained. Defects can originate from incorrect requirements, design mistakes, coding errors, integration problems or environmental issues. They can be categorized based on severity, priority, type, module and status. For example, an incorrect transaction amount in a banking application can be classified as a high-severity functional defect. The repository should contain details such as the defect description, steps to reproduce, expected result, actual result, severity, priority and resolution. Previous defect data can be analyzed to identify repeated patterns, and new test cases can be created based on these patterns. Using the defect repository effectively helps prevent recurring defects, improves test design and provides useful information for future testing.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures. Answer: Mobile Banking Application Crashes	3

	To identify the root cause of frequent crashes during fund transfer, the tester should first reproduce the problem under different conditions and check factors such as account balance, transaction amount, network connection, server response and application logs. Systematic test cases should include valid transactions, insufficient balance, invalid account details, large transaction amounts, network interruption and repeated transfers. For example, if the application crashes when the network is disconnected during a fund transfer, the problem may be related to improper error or exception handling. The defect should be recorded with the steps to reproduce, device details, application version and error information. After the developer fixes the issue, retesting and regression testing should be performed. This systematic approach helps identify the root cause and reduces the possibility of similar failures in future releases.	
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively. Answer: Testing and Debugging Testing and debugging are two different but related activities in software development. Testing is performed to identify defects or failures in the software, whereas debugging is performed by developers to find the cause of a defect and fix it. For example, if a banking application displays an incorrect account balance, the tester identifies the problem and reports it with proper evidence, while the developer debugs the code to find whether the calculation or database update is incorrect. In simple terms, testing mainly identifies what is wrong, while debugging finds why it is wrong and fixes it. If both activities are confused, defect detection and resolution may become ineffective. Therefore, testers should identify and report defects, developers should debug and fix them, and testers should perform retesting and regression testing after the fix.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage. Answer: Cross-Browser Testing	3

	Cross-browser testing is required to ensure that a web application works correctly on different browsers and versions. The testing strategy should include commonly used browsers such as Chrome, Edge, Firefox and Safari, along with different operating systems and screen sizes. Important functions such as login, payment, navigation, forms and page layout should be tested on each supported browser. Defects may occur because browsers use different rendering engines, JavaScript behavior or CSS support. For example, a payment button may work correctly in Chrome but may not be displayed properly in Firefox. Systematic cross-browser testing can identify such compatibility problems, while automation tools can improve test coverage. This approach ensures consistent application behavior and provides a better and more reliable user experience across different browsers.	
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy. Answer: Defect Classification in Payroll System The incorrect salary calculation in a payroll system can be classified mainly as a functional or logical defect because the software is not performing the required calculation correctly. It can also be classified based on severity and priority depending on its impact. For example, if a wrong logical condition causes incorrect deductions from an employee's salary, the final salary will be incorrect and the defect may have high severity. Understanding defect classes helps testers select suitable testing techniques such as equivalence partitioning, boundary value analysis and decision table testing. Code reviews and unit testing can help identify such errors early, while regression testing can ensure that the problem does not return in future releases. Proper defect classification therefore helps improve testing effectiveness and prevents repeated salary calculation problems.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness. Answer: Structured Test Case Design	3

	Random testing without proper test case design may miss important scenarios, so testing should be performed as a systematic engineering activity. First, the requirements should be analyzed and test conditions should be identified, after which test cases should be prepared with test ID, input data, test steps and expected results. Techniques such as equivalence partitioning, boundary value analysis, decision table testing and state transition testing can be used. For example, if an application accepts an age between 18 and 60, test cases should include values such as 17, 18, 19, 59, 60 and 61. Structured test cases provide better coverage and make testing repeatable and easier to maintain. Therefore, replacing random testing with proper test case design improves defect detection and overall testing effectiveness.	
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting. Answer: Data Inconsistency in Healthcare Application Data inconsistency occurs when different modules of an application show or store different values for the same information. Such defects may originate from incorrect database operations, synchronization problems, improper data validation or integration errors during development. For example, a patient's updated phone number may be correctly displayed in the profile module but the old number may still appear in the appointment module. Testers should compare data between related modules and verify database records using systematic integration and database testing. Test cases should cover data creation, modification, deletion and synchronization. Defects should be recorded with the affected module, input data, expected result and actual result. Proper defect tracking and reporting help maintain data integrity and improve the reliability of the healthcare application.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects. Answer: Defect Repository and Repeated Defects Repeated defects may occur because the company is maintaining a defect repository without properly analyzing the information stored in it. The repository should contain	3

	details such as defect type, affected module, severity, root cause, resolution and release information. Testers should analyze this historical data to identify common defect patterns. For example, if database-related defects repeatedly occur in the payment module, additional database validation and integration test cases should be added to the future test plan. Root cause analysis should also be performed to understand why the same type of defect is occurring repeatedly. Regression testing and review of previous defects should become part of the testing process. Thus, an effective defect repository not only records defects but also supports better test design and helps prevent recurring problems.	
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures. Answer: Web-Based Learning System During Peak Usage The failure of a web-based learning system during peak usage indicates that important performance scenarios were not properly tested. Testing gaps may include insufficient load testing, stress testing and concurrency testing. During test planning, the expected number of users and peak usage conditions should be identified, and the system should be tested with similar loads before deployment. For example, if thousands of students access an online examination system at the same time, the application should be tested under that condition. Response time, throughput, resource usage and system stability should be monitored during testing. Normal load, peak load and higher-than-expected load scenarios should also be included. Better test planning and performance coverage can identify bottlenecks early and improve the reliability of the system.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices. Answer: Role of Tester in SDLC Testers should be involved from the early stages of the SDLC instead of joining only after coding is completed. During requirement analysis, testers can review requirements and identify missing or unclear conditions. During design and development, they can prepare test scenarios and participate in reviews, and during implementation they can prepare test cases and test data. For example, if a requirement states that a user can withdraw money only up to a daily limit, the tester can identify boundary conditions at an early stage. Early involvement helps find	3

	defects before they become expensive to fix and reduces rework. Regular communication between testers, developers and other team members should be encouraged. This improves collaboration, software quality and the overall development process.	
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation. Answer: Incomplete Test Coverage Incomplete test coverage can cause a system to work correctly for some inputs but fail for other conditions. To identify coverage gaps, testers should review the requirements and map them to appropriate test cases. Techniques such as equivalence partitioning and boundary value analysis can be used to select meaningful test data. For example, if an application accepts marks from 0 to 100, testing only the value 50 is not enough; values such as 0, 1, 99, 100 and invalid values such as -1 and 101 should also be tested. Decision tables can be used when different combinations of conditions are involved. Requirement coverage, functional coverage and regression testing should be maintained. Proper test coverage increases confidence in the software and reduces failures after deployment.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards. Answer: Improving Defect Reporting Poor defect reporting can delay the identification and resolution of software problems, so defects should be reported in a clear and structured manner. A good defect report should include a defect ID, clear title, steps to reproduce, expected result, actual result, severity, priority and environment details. Screenshots or logs can also be attached when required. For example, instead of writing "Login not working," a clearer report would state that "Login fails when a valid username and password are entered and the system displays an invalid password message." Clear defect definitions help testers and developers understand the problem without confusion. A standard reporting	3

	format and centralized defect tracking system should be used. This improves communication, makes tracking easier and speeds up defect resolution.	
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency. Answer: Process-Based Testing Testing consistency across different modules can be improved by following a standardized, process-based testing approach. The process should define common activities such as requirement analysis, test planning, test case design, test execution, defect reporting, retesting and regression testing. For example, whether the module is related to login, payment or profile management, testers should follow the same test case structure and defect reporting format. Checklists, standard templates and test management tools can also help maintain consistency. Regular reviews should be conducted to make sure that testers follow the defined process. A standardized approach reduces differences between testers and modules, improves traceability and makes defect detection more effective. Therefore, process-based testing improves consistency and overall software quality.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design. Answer: Boundary Testing and Edge Cases Edge cases are often missed because testers mainly focus on normal inputs, so boundary testing is important for identifying defects near input limits. Boundary Value Analysis can be used by testing the minimum, maximum, just below and just above the allowed values. For example, if an ATM allows a withdrawal amount from ₹100 to ₹20,000, the tester should check ₹99, ₹100, ₹101, ₹19,999, ₹20,000 and ₹20,001. These test cases can identify errors in validation and boundary conditions. Edge cases should be included during test case design and should also be considered	3

	during regression testing. Proper boundary testing improves test coverage and reduces the possibility of unexpected failures after deployment.	
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices. Answer: Defect Repository and Documentation Proper defect documentation is important for tracking and managing software problems throughout the development process. A defect repository should maintain information such as defect ID, description, affected module, steps to reproduce, expected result, actual result, severity, priority, assigned person and status. For example, a database defect found in the payment module can be recorded along with its error message and reproduction steps so that developers can understand and fix it easily. A structured repository also helps identify repeated defects and common root causes across different releases. Standard templates, proper categorization and regular repository reviews should be followed. Good documentation improves communication, traceability and defect management and helps prevent similar defects in future releases.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection. Answer: Redesigning Testing for Banking Transactions Repeated failures in banking transaction processing indicate that the existing testing process is not systematic enough. The testing approach should begin with requirement analysis and identification of important transaction scenarios. Test cases should cover successful transactions, insufficient balance, invalid account details, duplicate transactions, cancelled transactions, network failures and transaction limits. Techniques such as equivalence partitioning, boundary value analysis, decision table testing and regression testing should be applied. For example, when ₹5,000 is transferred from one account to another, the sender's balance should decrease by ₹5,000 and the receiver's balance should increase by ₹5,000, and if the transaction fails, the system should not partially update the accounts. Defects should be stored in	3

a repository and analyzed for recurring patterns, while automated regression testing should be used for critical transactions. This structured approach improves defect detection, reduces repeated failures and increases the reliability of the banking system.	
--	--



Code of Conduct:

*I **Mugesh S (192411055)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S. Mugesh.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand